CHAPTER 4

Er. PIYUSH PANT

MAHENDRA RATNA CAMPUS

INHERITANCE

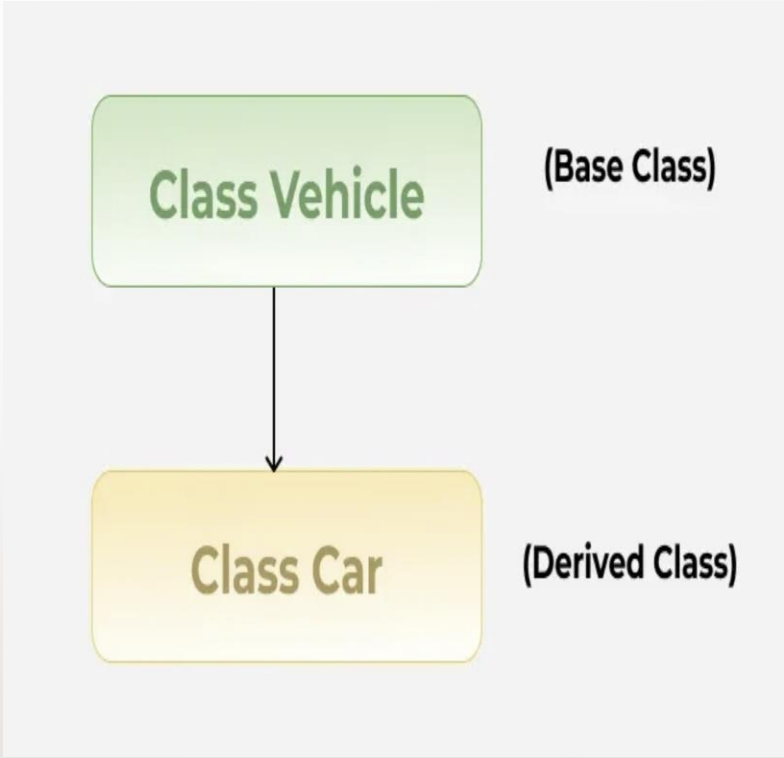
```
class Base {
  // base class members
};

class Derived : access_specifier Base {
  // additional members of derived class
};
```

- **Inheritance** is a feature of Object-Oriented Programming (OOP) in C++ that allows a class (called the **derived class**) to inherit properties and behaviors (data members and member functions) from another class (called the **base class**).

Type of Inheritance	Description	Syntax Example
Single Inheritance	One derived class inherits from one base class.	class B : public A {}
Multiple Inheritance	One derived class inherits from more than one base class.	class C : public A, public B {}
Multilevel Inheritance	A class is derived from a derived class.	class C : public B {}
Hierarchical Inheritance	Multiple classes are derived from the same base class.	class B : public A {}; class C : public A {};
Hybrid Inheritance	Combination of two or more types of inheritance.	Uses virtual base classes to avoid ambiguity.

SINGLE INHERITANCE

Type of inheritance when child class is derived from single base class .

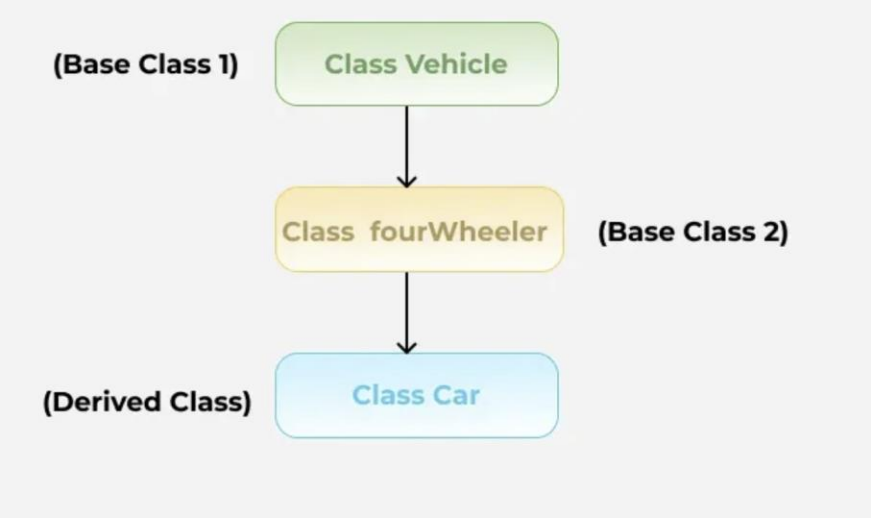


```
#include<iostream>
using namespace std;
class Animal {
public:
    void eat() {
        cout << "Eating..." << endl;
    }
};
class Dog : public Animal {
public:
    void bark() {
        cout << "Barking..." << endl;
    }
};
```

```
int main() {
    Dog d;
    d.eat();   // inherited
    d.bark();  // own function
    return 0;
}
```

MULTI-LEVEL INHERITANCE

- **Multilevel Inheritance** is a type of inheritance in which a class is derived from another **derived class**, forming a chain of inheritance.
- Class B inherits from class A.
- Class C inherits from class B.
- Thus, **class C indirectly inherits** from class A.



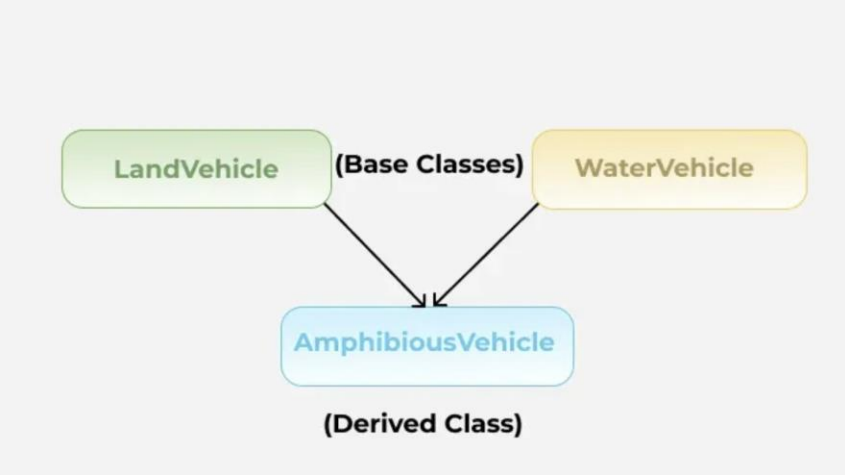

```
class A {
  // base class
};

class B : public A {
  // derived from A
};

class C : public B {
  // derived from B, and
indirectly from A
};
```

EXAMPLE

```
#include<iostream>
using namespace std;
class Animal {
public:
   void eat() {
      cout << "Eating..." << endl;
   }
};
class Dog : public Animal {
public:
   void bark() {
      cout << "Barking..." << endl;
   }
};
class Puppy : public Dog {
public:
   void weep() {
      cout << "Weeping..." << endl;
   }
};

int main() {
   Puppy p;
   p.eat();  // from Animal
   p.bark();  // from Dog
   p.weep();  // own function
   return 0;
}
```

MULTIPLE INHERITANCE

- **Multiple Inheritance** is a feature in C++ where a single derived class inherits from **two or more base classes**.
- It allows the derived class to combine functionalities from multiple base classes.




```
class Base1 {
  // members
};

class Base2 {
  // members
};

class Derived : public Base1, public Base2 {
  // combined members from Base1 and Base2
};
```

AMBIGUITY IN MULTIPLE INHERITANCE

If two base classes have a function with the same name, the compiler doesn't know which one to call.

```
class A {
public:
   void show() {
      cout << "A::show()" << endl;
   }
};

class B {
public:
   void show() {
      cout << "B::show()" << endl;
   }
};

class C : public A, public B {};
```

```
int main() {
   C obj;
   obj.show();  // ❌ Error: Ambiguous
}
```

SOLUTION

Specify class name explicitly

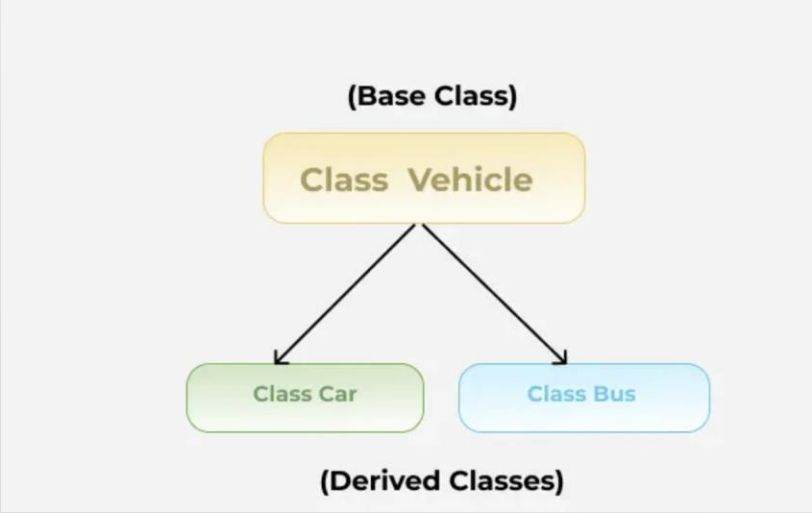
obj.A::show();

obj.B::show();

```
int main() {
   C obj;
   obj.A::show();
   Obj.B::show();
}
```

HIERARCHAL INHERITANCE

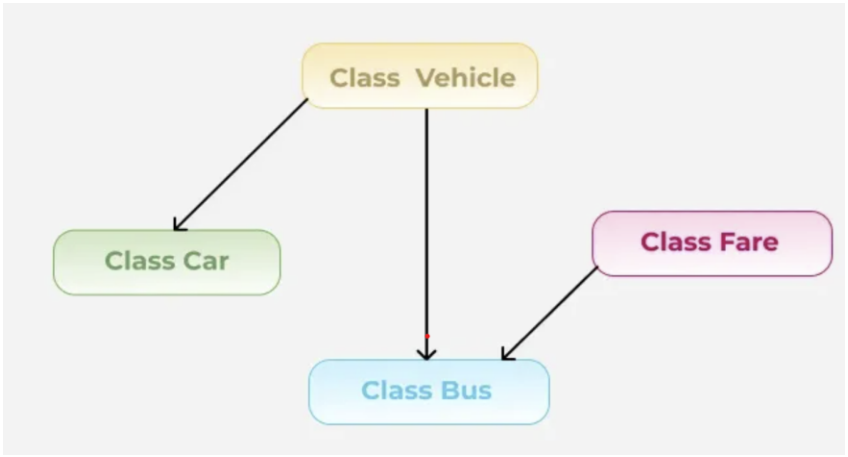
- In **Hierarchical Inheritance**, **multiple derived classes** inherit from a **single base class**.
- One base class acts as a parent.
- Two or more classes directly inherit from this single base class.




```
class Base {
  // base class members
};

class Derived1 : public Base {
  // members of derived class 1
};

class Derived2 : public Base {
  // members of derived class 2
};
```

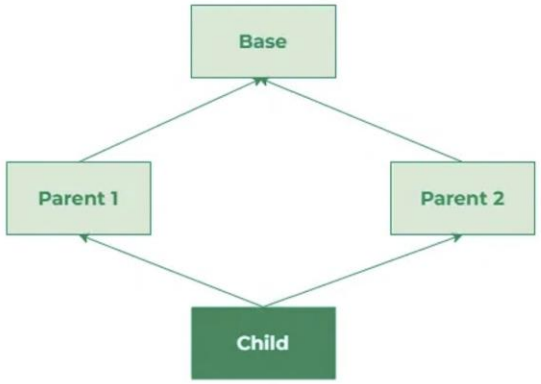
HYBRID INHERITANCE

- **Hybrid Inheritance** is a combination of two or more types of inheritance (such as single, multiple, and multilevel) in a single program.
- It merges features of multiple inheritance forms.
- It can lead to the **Diamond Problem**, which can be solved using **virtual inheritance**.
- Example of multiple and hierarchical inheritance combined.



DIAMOND PROBLEM

- The Diamond Problem is an ambiguity error that arises in multiple inheritance when a derived class inherits from two or more base classes that share a common ancestor.
- This results in the inheritance hierarchy forming a diamond shape, hence the name "Diamond Problem."
- The ambiguity arises because the derived class has multiple paths to access members or methods inherited from the common ancestor, leading to confusion during method resolution and member access.
- **Without virtual inheritance, the compiler treats each path to the common base as a separate instance**, meaning the derived class ends up with multiple independent copies of the base class. This duplication causes ambiguity, wastes memory, and can lead to inconsistent behavior when accessing inherited members.



- **C++ addresses the Diamond Problem using virtual inheritance.**
- **Virtual inheritance ensures that there is only one instance of the common base class, eliminating the ambiguity.**

DERIVING CLASS USING ACCESS SPECIFIERS: PRIVATE, PUBLIC AND PROTECTED

- In C++, when you inherit from a base class you can specify the inheritance mode—`public`, `protected`, or `private`. This choice controls how the base class's members (public, protected, private) are treated in the derived class.

The inheritance specifier determines the new access level of those inherited members in the derived class.

Inheritance Mode	Base's Public Members	Base's Protected Members	Base's Private Members
public	public	protected	inaccessible
protected	protected	protected	inaccessible
private	private	private	inaccessible

METHOD OVERRIDING

Method Overriding occurs when a **derived class** provides its own implementation of a **function that is already defined in its base class** with the same name, return type, and parameters.

It allows a derived class to modify or extend the behavior of a base class function.

Supports **runtime polymorphism** (dynamic binding).

METHOD OVERRIDING

- The function in the base class should be declared as **`virtual`** for overriding.
- The overriding function in the derived class **replaces** the base class function when accessed through a base class pointer or reference.
- If base class function is not virtual, overriding will result in **function hiding** (compile-time binding).

WHY METHOD OVERRIDING

- Methods are selected at runtime based on object type.
- Common interfaces and implementations can be reused as we only need to implement the interface for base class.
- Changes in base classes automatically affect derived classes.
- Facilitates the implementation of design patterns different design patterns.
- Client code interacts with base class interface, not specific implementations reducing the coupling.

LIMITATIONS

- While runtime function overriding provides numerous advantages, it also comes with certain limitations:
- Virtual function calls are generally slower than non-virtual function calls due to the indirection involved in the virtual table lookup.
- Implementing and maintaining polymorphic code can be more complex compared to simpler non-polymorphic code.
- Virtual tables and virtual pointers add to the memory overhead, especially in large systems with many classes.
- Errors related to function overriding (e.g., incorrect signatures) might only be detected at runtime, leading to potential bugs.

CONTAINERSHIP

Containership is the process of embedding one class inside another by declaring an object of the contained class as a member of the container class.

This allows the container class to use the functionality of the contained class without inheriting from it.

```
class Contained {
public:
    void show() {
        cout << "Inside Contained class" << endl;
    }
};

class Container {
private:
    Contained obj;  // Containership
public:
    void display() {
        obj.show();  // Accessing contained class method
    }
};
```

CHARACTERISTICS OF CONTAINERSHIP

Encapsulation: Promotes modular design by grouping related functionality.

Code Reusability: Reuses existing class functionality without inheritance.

Constructor Chaining: The constructor of the contained class is automatically called when the container class is instantiated.

Access Control: Only public members of the contained class are accessible from the container class.

CONSTRUCTOR EXECUTION ORDER

When you create an object of a derived class, C++ ensures that all its sub-objects are initialized before the derived-class constructor body executes.

1. Base class constructors
2. Member objects of the derived class (in declaration order)
3. Derived class constructor body

Multiple Inheritance

If a class inherits from more than one base:

1. Base constructors are called **left-to-right** as listed in the inheritance clause.
2. Member objects follow.
3. Finally, the derived class constructor is invoked.

Virtual Inheritance

When virtual base classes are involved:

1. Virtual base class constructors run **before** all non-virtual bases, regardless of their position in the inheritance list

DESTRUCTOR EXECUTION ORDER

Destructors run in the exact reverse order of constructors to safely tear down resources:

1. Derived class destructor
2. Member objects (in reverse declaration order)
3. Base class destructors (in reverse inheritance order)